

Lebanese International University

Computer Vision and Assistive Technology

Computer Vision Glasses

for People with Visual Impairments

Project Report



Camera images → visual recognition → spoken feedback

Author: Jaber Mobarak

Project year: 2024

Source code and demonstration

[GitHub project repository](#)

https://github.com/jabermobarak/Computer_vision_Glasses_for_blind_people

The

cover illustration is conceptual and is not a photograph of the prototype.

Abstract

This project explores an assistive glasses concept that converts visual information into spoken feedback for people with visual impairments. The implemented Python program retrieves images from a network camera, decodes and resizes them using OpenCV, and provides two selectable processing modes. The object-recognition mode uses cvlib to identify common objects and announce their labels. The text-recognition mode uses EasyOCR to extract English text and pyttsx3 to read it aloud. A graphical window and keyboard controls support operation during development.

The repository also contains an Arduino sketch for a joystick-controlled counter, seven-segment display, LED, and buzzer. Its behavior is documented separately because the supplied files do not establish a communication link between that sketch and the Python recognition program. The report explains the software architecture, processing sequence, user controls, peripheral logic, and implementation constraints. It also defines a future evaluation procedure without presenting unmeasured accuracy, latency, or user-study results.

Keywords: computer vision; assistive technology; object recognition; optical character recognition; speech synthesis; OpenCV; EasyOCR; Arduino.

Project information

Author	Jaber Mobarak
University	Lebanese International University
Project year	2024
Project type	Assistive technology prototype and computer vision software
Source	GitHub project repository

Scope of this report

The technical account is grounded in the repository’s `objecreco.py`, `sketch_apr3a.ino`, and README. The year 2024 identifies the project; the repository was subsequently updated. Included source listings reproduce the supplied implementation rather than an improved version.

Contents

1	Introduction and Objectives	1
2	System Architecture	2
3	Computer Vision Implementation	3
4	Interaction and Arduino Peripheral Logic	4
5	Operation and Reproducibility	6
6	Evaluation Plan and Implementation Limits	7
7	Conclusion	8
	Source References	8
A	Python Source Code	10
B	Arduino Source Code	11

1 Introduction and Objectives

1.1 Motivation

Visual information is central to many everyday activities, including identifying objects and reading labels or printed text. This project investigates whether a camera-based prototype can present some of that information through speech. The aim is to demonstrate a practical computer vision workflow for accessibility, with recognition software that can form part of a wearable concept.

The prototype announces object labels and recognized text. It does not estimate safe walking routes, detect all hazards, measure obstacle distance, or provide validated navigation guidance. Its intended contribution is access to selected visual information.

1.2 Project objectives

1. Retrieve images continuously from a network-accessible camera endpoint.
2. Decode image bytes and prepare frames for visual processing.
3. Provide a selectable mode for recognizing common objects.
4. Provide a selectable mode for reading English text in images.
5. Convert recognition outputs into audible messages.
6. Demonstrate peripheral control logic with an Arduino sketch.

1.3 Software contribution

The computer vision component combines camera acquisition, OpenCV image processing, cvlib object recognition, EasyOCR text recognition, and pyttsx3 speech output in a single Python program. It includes mutually exclusive recognition modes and a development interface for viewing the image stream. The Arduino component demonstrates additional input and output handling independently of the vision program.

Table 1: Implemented functions and their evidence in the source files.

Function	Implementation evidence
Camera acquisition	HTTP image retrieval using <code>urllib.request.urlopen</code> .
Image preparation	NumPy byte conversion, OpenCV decoding, resizing to 320×240 .
Object recognition	<code>cv.detect_common_objects</code> and bounding-box rendering.
Text recognition	<code>easyocr.Reader(['en'])</code> and <code>readtext</code> .
Spoken feedback	<code>pyttsx3</code> with synchronous speech execution.
Mode selection	Keyboard keys O, T, and Q in the OpenCV window.
Peripheral behavior	Arduino joystick, counter, display, LED, and buzzer logic.

2 System Architecture

2.1 Implemented vision pipeline

The Python program runs on a host computer. It requests camera images over the network and performs recognition and speech synthesis on the host. The code does not execute recognition on an ESP32. The README mentions Arduino and ESP32 hardware, but ESP32 camera firmware and a wiring diagram are not included.

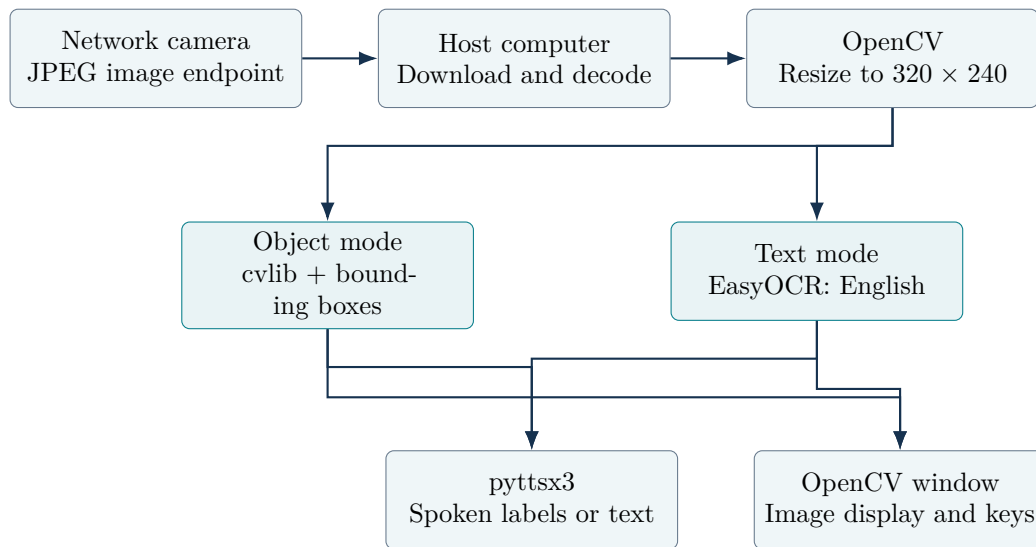


Figure 1: The implemented host-based processing architecture. Object and text modes are selected individually; speech and display are executed sequentially in the main loop.

2.2 Functional layers

Layer	Responsibility
Acquisition	Request the current JPEG image from the configured camera URL.
Preparation	Convert received bytes into a decoded, resized image.
Recognition	Apply either object recognition or English OCR.
Output	Speak the recognition result and display the processed frame.
Interaction	Handle recognition-mode toggles and the quit command.

2.3 Arduino subsystem boundary

The Arduino sketch has its own input-processing and output-update loop. There is no serial, network, or other communication code connecting it to the Python script. Consequently, the joystick cannot be described as controlling the recognition modes on the basis of the supplied files.

3 Computer Vision Implementation

3.1 Camera acquisition and preprocessing

The camera endpoint is configured in the variable `url`, ending in `/cam-hi.jpg`. Each loop iteration downloads an image, converts the byte buffer into a NumPy array, decodes it with OpenCV, and resizes it to 320×240 pixels. The configured address belongs to the development network and must be changed for another deployment.

Let B_t denote the downloaded bytes at iteration t . The input to recognition is

$$I_t = \text{Resize}_{320 \times 240}(\text{Decode}(B_t)). \quad (1)$$

Each resized image contains $320 \cdot 240 = 76,800$ pixels. Resizing can reduce the workload relative to processing a larger image, while also reducing the detail available for small objects or fine text. The original camera resolution and the resulting speed difference are not measured in the repository.

3.2 Object-recognition mode

The function `object_recognition(img)` calls `cv.detect_common_objects(img)`. It obtains bounding boxes, class labels, and confidence values, then draws bounding boxes with `draw_bbox`. The labels are joined into a single spoken sentence beginning with “I see”.

Represent the detector output as

$$\mathcal{D}_t = \{(b_i, \ell_i, c_i)\}_{i=1}^{n_t}, \quad (2)$$

where b_i is a bounding box, ℓ_i is an object label, and c_i is its confidence. Although the script receives confidence values, it does not apply its own confidence threshold or announce them. Detector-specific defaults depend on the installed cvlib version; the project does not pin that version or explicitly select a detector model.

The program neither removes repeated labels nor tracks objects across frames. If no objects are detected, the label string is empty and the code still invokes speech with the prefix “I see”. This behavior should be refined before wider use.

3.3 Text-recognition mode

The OCR reader is configured for English using `['en']`. The function `text_recognition` calls `reader.readtext(img)` and joins the recognized strings in the order returned by EasyOCR:

$$T_t = \text{Join}(s_1, s_2, \dots, s_{m_t}), \quad (3)$$

where s_j is the text field of OCR result j . If the combined text is nonempty, the program reads it aloud with the prefix “I can read”. Otherwise, it announces that no text was found. OCR confidence values and text-region coordinates are not used by the application after extraction.

This implementation reads English text. It does not translate recognized text or provide implemented language switching. Although `googletrans.Translator` is imported and instantiated, it is not called by either recognition function.

3.4 Speech synthesis

The `speak(text)` function passes text to `engine.say(text)` and executes `engine.runAndWait()`. This is a blocking call: the image-processing loop waits until speech execution completes. The

program selects the voice at index 1 of the host’s available voices, without checking whether that index exists or specifying a language explicitly.

3.5 Processing schedule

The parameter `frame_skip` is set to 1. Therefore every downloaded frame is eligible for recognition when a mode is active. This does not guarantee a particular camera frame rate, because each image is acquired through a separate request and speech can delay the next iteration.

A conceptual per-iteration time budget is

$$\tau_t = \tau_{\text{network}} + \tau_{\text{decode/resize}} + \tau_{\text{recognition}} + \tau_{\text{speech}} + \tau_{\text{UI}}. \quad (4)$$

This equation explains the sequential workload; it is not a measured performance result. In a disabled-recognition state, recognition and speech are skipped.

Table 2: Configuration values visible in the Python implementation.

Parameter	Value or behavior
Processing image size	320 × 240 pixels
Frame-skip interval	1; process every acquired frame when a mode is active
OCR language	English, configured as <code>['en']</code>
Initial recognition state	Both object and text recognition disabled
Voice selection	Second voice in the host voice list, <code>voices[1]</code>
Keyboard polling	<code>cv2.waitKey(5)</code> ; not an end-to-end latency guarantee
Translation	Translator initialized but unused

4 Interaction and Arduino Peripheral Logic

4.1 Recognition-mode controls

The OpenCV window displays camera frames. At startup, neither recognition mode is enabled. Pressing O toggles object recognition and disables text recognition. Pressing T toggles text recognition and disables object recognition. Pressing Q terminates the loop and closes the OpenCV windows.

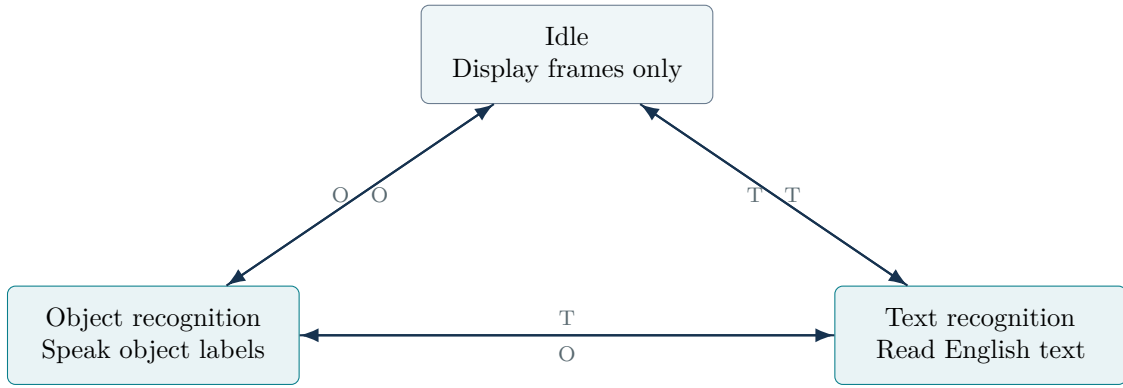
4.2 Arduino pin assignments

4.3 Peripheral sequence

During setup, the buzzer plays a 1000 Hz tone for one second. The sketch configures output pins and initializes the seven-segment display to zero. In each loop, it reads the joystick axes and button. A pressed button resets the counter and starts a two-second LED blink sequence. Otherwise, vertical movement increments the counter and horizontal movement decrements it. Both up and down movements increment; both left and right movements decrement. This is the actual logic of the supplied sketch.

The displayed digit is computed using integer division:

$$d = \left\lfloor \frac{C}{5} \right\rfloor, \quad (5)$$



Q exits from any state. Both recognition modes are mutually exclusive.

Figure 2: Mode transitions implemented by the Python keyboard controls.

Table 3: Connections defined by the Arduino sketch. Actual wiring is not documented in the repository.

Peripheral	Assigned pins
Buzzer	Digital pin 11
LED	Digital pin 12
Joystick button	Digital pin 10; INPUT_PULLUP
Joystick horizontal axis	Analog pin A0
Joystick vertical axis	Analog pin A1
Seven-segment display	Segments A–G on digital pins 2–8

Table 4: Joystick decisions in the Arduino loop, in priority order.

Condition	Action
Button reads LOW	Reset counter to 0; update display; blink LED.
$y > 800$	Increment counter, capped at 50.
$y < 200$	Increment counter, capped at 50.
$x > 800$	Decrement counter, bounded below by 0.
$x < 200$	Decrement counter, bounded below by 0.
No condition satisfied	Leave counter unchanged.

where C is the counter. The display routine accepts only digits 0–9. At $C = 50$, it receives $d = 10$ and returns without updating the display; the previous digit remains. The sketch also includes a 200 ms loop delay and blocking LED-blink delays. These features belong to a peripheral demonstration, and no role in camera or recognition control is established in the files.

5 Operation and Reproducibility

5.1 Required components

The vision program requires a host computer with Python, a reachable camera image endpoint, audio output, and a graphical environment for the OpenCV window. The README identifies Arduino and ESP32 as project hardware. To recreate the complete physical setup, additional camera firmware, wiring, and assembly information would be required.

Software component	Purpose
Python	Executes the camera, recognition, and speech program.
OpenCV (<code>cv2</code>)	Image decoding, resizing, display, and keyboard input.
NumPy	Converts the downloaded image buffer into an array.
cvlib	Common-object detection and bounding-box visualization.
EasyOCR	English optical character recognition.
pyttsx3	Host-based speech synthesis.
googletrans	Imported in the supplied program, although translation is unused.
Arduino toolchain	Builds and uploads the separate peripheral sketch.

The repository does not contain a dependency lockfile, Python version specification, or complete environment instructions. Model downloads and backend requirements can depend on library versions. This report therefore documents the observed workflow without claiming a verified installation command for every platform.

5.2 Execution procedure

1. Configure the network camera so that its JPEG endpoint is reachable from the host.
2. Install a compatible environment for the packages imported in `objecreco.py`.
3. Replace the script's camera URL with the address of the actual camera endpoint.
4. Check that the selected speech voice exists on the host and audio output works.
5. Run `python objecreco.py`. The initial state displays camera frames only.
6. Focus the OpenCV window and press O to recognize objects, or T to read text.
7. Press the active mode's key again to disable recognition, or press Q to exit.

5.3 Verification procedure

Table 5 provides checks derived from the implementation. These are proposed checks, not recorded successful test results.

Table 5: Suggested functional verification cases.

Check	Procedure	Behavior to verify
Image acquisition	Start with a reachable camera URL.	Decoded frames appear in the window.
Object mode	Press O with a common object visible.	The detector output is displayed and spoken.
Text mode	Press T with clear English text visible.	Recognized text is spoken.
Mode exclusivity	Switch from O to T and back.	Only one recognition mode remains active.
No text	Use an image without readable text in text mode.	The no-text message is spoken.
Quit	Press Q after processing completes.	The loop ends and windows close.
Arduino bound-ary	Increment the counter to 50.	The display routine rejects digit 10.

6 Evaluation Plan and Implementation Limits

6.1 Evidence available

The repository provides source code and a demonstration link. It does not provide an annotated evaluation set, stored detection outputs, OCR reference transcripts, timing measurements, or participant feedback records. Accordingly, no numerical accuracy, frame-rate, latency, or accessibility-improvement claims are made in this report.

6.2 Proposed quantitative evaluation

Object recognition can be evaluated on labeled camera images containing the intended classes and viewing conditions. For bounding-box detection, predictions should be matched to reference boxes using a stated overlap threshold and class agreement. Then precision and recall can be calculated as

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}, \quad (6)$$

with undefined cases handled explicitly. These are proposed evaluation measures, not reported project scores.

For OCR, compare output text to reference transcripts and report character error rate:

$$\text{CER} = \frac{S + D + I}{N}, \quad (7)$$

where S , D , and I count character substitutions, deletions, and insertions, and N is the reference length. Define text normalization before measuring performance, and treat empty references separately.

Response timing should distinguish time to first audible feedback from total cycle time. The latter includes speech duration because the current implementation blocks while speaking. Record the host specifications, camera setup, input resolution, software versions, and lighting conditions alongside every measurement.

6.3 Current limitations

- **Speech blocks acquisition:** long messages delay the next image request and keyboard handling.
- **Repeated messages:** no cooldown or cross-frame deduplication suppresses repeated object labels or text.
- **Limited error handling:** the code lacks explicit recovery for failed requests, invalid images, OCR failures, and missing voices.
- **English OCR only:** multilingual recognition, translation, and language switching are not implemented.
- **Development controls:** operation depends on a graphical window and keyboard rather than a verified wearable interface.
- **Incomplete hardware documentation:** camera firmware, power design, wiring, assembly, and Python–Arduino communication are absent.
- **No measured evaluation:** recognition reliability and usability with intended users have not been established by the supplied evidence.

6.4 Development priorities

The next software step is to separate acquisition, recognition, and speech into coordinated tasks, with a bounded queue to prevent obsolete messages from accumulating. Recognition outputs should be filtered and repeated announcements controlled. Camera timeouts, decode checks, and voice fallbacks should be added. A physical mode selector requires an explicit communication protocol and documented integration. Later extensions could add supported OCR languages and translation, followed by controlled evaluation and accessibility testing with consenting participants.

7 Conclusion

The project demonstrates a host-based visual-to-speech pipeline for an assistive glasses concept. A Python application retrieves network camera images, performs common-object recognition or English OCR, and speaks the results. Its implementation brings together camera acquisition, image processing, recognition libraries, and speech synthesis in a compact prototype.

The source also provides a separate Arduino peripheral demonstration. Its connection to the vision pipeline is not implemented in the supplied files. Further work should focus on robust runtime behavior, nonblocking speech, accessible physical controls, complete hardware documentation, and measured recognition and usability evaluation. These steps would provide a stronger basis for assessing the prototype’s practical value.

Source References

1. Jaber Mobarak. *Computer_vision_Glasses_for_blind_people*. Project repository.
https://github.com/jabermobarak/Computer_vision_Glasses_for_blind_people
2. Python implementation: `objecreco.py`.
[View the Python source on GitHub.](#)

3. Arduino implementation: `sketch_apr3a.ino`.
[View the Arduino source on GitHub.](#)
4. Project README and linked demonstration.
[View the README on GitHub.](#)

The appendices reproduce the analyzed repository files. The project year remains 2024; the repository includes a later README update.

A Python Source Code

The original camera address appears in this listing. It is a local development endpoint and must be changed when running the program on another network. Imports and existing behavior are reproduced without modification.

```

1  import cv2
2  import cvlib as cv
3  from cvlib.object_detection import draw_bbox
4  import urllib.request
5  import numpy as np
6  import pyttsx3
7  import easyocr
8  from googletrans import Translator
9
10 url = 'http://172.20.14.6/cam-hi.jpg'
11 frame_skip = 1 # Process every frame
12
13 # Initialize EasyOCR
14 reader = easyocr.Reader(['en'])
15
16 # Initialize voice engine
17 engine = pyttsx3.init()
18 voices = engine.getProperty('voices')
19 engine.setProperty('voice', voices[1].id) # Change index to change the voice (0 for male, 1 for female)
20
21 # Initialize Google Translate API
22 translator = Translator()
23
24 def speak(text):
25     engine.say(text)
26     engine.runAndWait()
27
28 def object_recognition(img):
29     bbox, label, conf = cv.detect_common_objects(img)
30     img = draw_bbox(img, bbox, label, conf)
31     objects = ", ".join(label)
32     speak(f"I see {objects}")
33     return img
34
35 def text_recognition(img):
36     results = reader.readtext(img)
37     detected_text = ' '.join([result[1] for result in results])
38     if detected_text:
39         speak(f"I can read: {detected_text}")
40     else:
41         speak("I couldn't find any text.")
42     return img
43
44 def run():
45     frame_count = 0
46     object_recognition_enabled = False
47     text_recognition_enabled = False
48
49     cv2.namedWindow("detection", cv2.WINDOW_AUTOSIZE)
50     while True:
51         img_resp = urllib.request.urlopen(url)
52         imgnp = np.array(bytearray(img_resp.read()), dtype=np.uint8)
53         im = cv2.imdecode(imgnp, -1)
54         im = cv2.resize(im, (320, 240)) # Resize image for faster processing
55
56         if frame_count % frame_skip == 0:
57             if object_recognition_enabled:
58                 im = object_recognition(im)
59             elif text_recognition_enabled:
60                 im = text_recognition(im)
61
62         cv2.imshow('detection', im)
63         key = cv2.waitKey(5)
64         if key == ord('q'):
65             break
66         elif key == ord('o'):
67             object_recognition_enabled = not object_recognition_enabled
68             text_recognition_enabled = False
69         elif key == ord('t'):
70             text_recognition_enabled = not text_recognition_enabled
71             object_recognition_enabled = False
72
73         frame_count += 1
74
75     cv2.destroyAllWindows()
76
77 if __name__ == '__main__':
78     print("started")
79     run()

```

B Arduino Source Code

This sketch is reproduced independently of the Python application. No communication link between them is defined in the supplied source.

```

1 // Pin definitions
2 const int buzzerPin = 11;
3 const int ledPin = 12;
4 const int buttonPin = 10;
5 const int xAxisPin = A0;
6 const int yAxisPin = A1;
7
8 // Seven-segment display segment pins
9 const int A = 2;
10 const int B = 3;
11 const int C = 4;
12 const int D = 5;
13 const int E = 6;
14 const int F = 7;
15 const int G = 8;
16
17 const int segmentPins[7] = {A, B, C, D, E, F, G};
18
19 // Counter variable
20 int counter = 0;
21
22 // Segment display digits 0-9 using the provided pattern
23 const byte seven_seg_digits[10][7] = {
24   { 1, 1, 1, 1, 1, 1, 0 }, // = 0
25   { 0, 1, 1, 0, 0, 0, 0 }, // = 1
26   { 1, 1, 0, 1, 1, 0, 1 }, // = 2
27   { 1, 1, 1, 1, 0, 0, 1 }, // = 3
28   { 0, 1, 1, 0, 0, 1, 1 }, // = 4
29   { 1, 0, 1, 1, 0, 1, 1 }, // = 5
30   { 1, 0, 1, 1, 1, 1, 1 }, // = 6
31   { 1, 1, 1, 0, 0, 0, 0 }, // = 7
32   { 1, 1, 1, 1, 1, 1, 1 }, // = 8
33   { 1, 1, 1, 0, 0, 1, 1 } // = 9
34 };
35
36 void setup() {
37   // Set up the buzzer
38   pinMode(buzzerPin, OUTPUT);
39   // Play the piezo buzzer for one second
40   tone(buzzerPin, 1000); // 1000 Hz tone
41   delay(1000);
42   noTone(buzzerPin);
43
44   // Set up the LED
45   pinMode(ledPin, OUTPUT);
46
47   // Set up the joystick button
48   pinMode(buttonPin, INPUT_PULLUP);
49
50   // Set up the seven-segment display pins
51   for (int i = 0; i < 7; i++) {
52     pinMode(segmentPins[i], OUTPUT);
53   }
54
55   // Initialize the seven-segment display
56   displayDigit(0);
57 }
58
59 void loop() {
60   // Read the joystick positions
61   int xValue = analogRead(xAxisPin);
62   int yValue = analogRead(yAxisPin);
63   bool buttonPressed = digitalRead(buttonPin) == LOW;
64
65   if (buttonPressed) {
66     // Reset counter to 0 and blink LED for 2 seconds
67     counter = 0;
68     displayDigit(0);
69     blinkLED(2000);
70   } else {
71     // Joystick movement handling
72     if (yValue > 800) { // Up
73       incrementCounter();
74     } else if (yValue < 200) { // Down
75       incrementCounter();
76     } else if (xValue > 800) { // Right
77       decrementCounter();
78     } else if (xValue < 200) { // Left
79       decrementCounter();
80     }
81   }
82
83   delay(200); // Debounce delay
84 }
85
86 void incrementCounter() {

```

```
87     if (counter < 50) {
88         counter++;
89         displayDigit(counter / 5);
90     }
91 }
92
93 void decrementCounter() {
94     if (counter > 0) {
95         counter--;
96         displayDigit(counter / 5);
97     }
98 }
99
100 void displayDigit(int num) {
101     // Display a number on the seven-segment display
102     if (num < 0 || num > 9) return;
103
104     for (int i = 0; i < 7; i++) {
105         digitalWrite(segmentPins[i], seven_seg_digits[num][i]);
106     }
107 }
108
109 void blinkLED(int duration) {
110     // Blink the LED for the given duration (in milliseconds)
111     unsigned long endTime = millis() + duration;
112     while (millis() < endTime) {
113         digitalWrite(ledPin, HIGH);
114         delay(500);
115         digitalWrite(ledPin, LOW);
116         delay(500);
117     }
118 }
```